How to ship a registry when nobody wants to run one

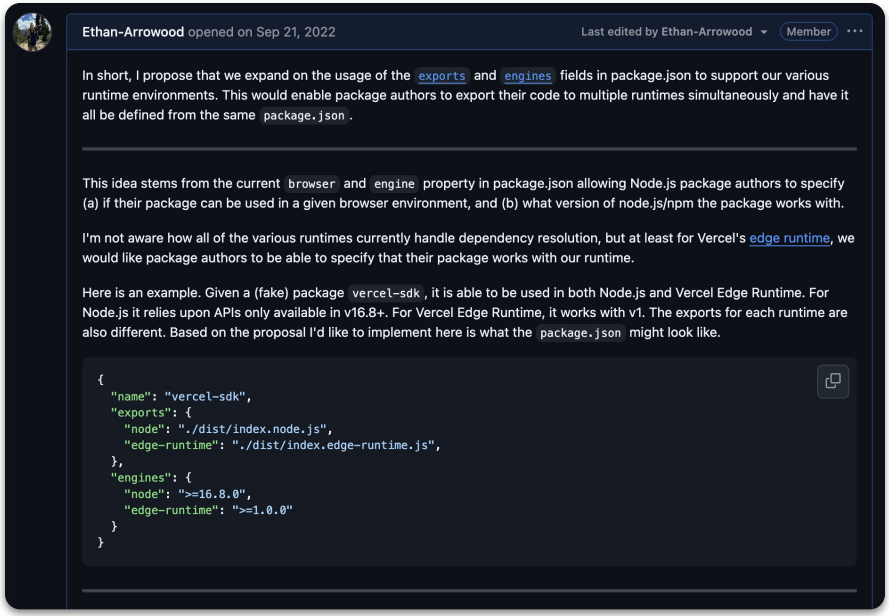
Aki Rose Braun & Ethan Arrowood

September 2022

One package, multiple runtimes

Proposal: expand usage of package.json `exports` and `engines` field to support various runtime environments.

Projects can specify exactly what runtimes they support.



Source: https://github.com/WinterTC55/admin/issues/36

October 2022

Introducing: `runtime-keys` proposal



W3C Community Group Draft Report

Runtime Keys

Draft Community Group Report, 19 October 2022

▼ More details about this document

This version:
http://runtime-keys.proposal.wintercg.org

Issue Tracking:
GitHub

Editor:
Ethan Arrowood (Vercel)

Copyright © 2022 the Contributors to the Runtime Keys Specification, published by the Web-interoperable Runtimes Community Group under the W3C Community Contributor License Agreement (CLA). A human-readable summary is available.

TABLE OF CONTENTS

1 Introduction
1.1 Example Usage

2 Keys
2.1 Alibaba Cloud - edge-routine
2.2 Cloudflare - workerd
2.3 Deno Land - Deno
2.4 Lagon - Lagon Runtime
2.5 Meta - React Native
2.6 Netlify - Edge Functions
2.7 OpenJS Foundation - Electron
2.8 OpenJS Foundation - Node.js
2.9 Oven - Bun
2.10 Vercel - Edge Light

3 Key Management
3.1 Adding a key
3.2 Modifying a key

Conformance
Document conventions

References
Normative References

Abstract

A proposal defining standard identifier keys for various runtime environments

Status of this document

This specification was published by the Web-interoperable Runtimes Community Group. It is not a W3C Standard nor is it on the W3C Standards Track. Please note that under the W3C Community Contributor License Agreement (CLA) there is a limited opt-out and other conditions apply. Learn more about W3C Community and Business Groups.

§ 1. Introduction

This proposal defines a list of keys that represent various runtime environments.

These keys can be used in a variety of ways, and should be considered a reliable representation of the given runtime.

For example, they may be used in a project configuration file to indicate the given project supports the specified runtime.

This specification does not detail *how* the keys can be used; that is left up to community developed tools.

The point of this proposal is to only define *what* the keys are in order to prevent conflicts, and provide users a reliable list of platforms they can build tooling around.

Inclusion in this registry does not imply that the specified runtime is conformant with the WinterCG Minimum Common API.

Inclusing in this registry does not imply endorsement of any kind.



Ethan-Arrowood commented on Oct 19, 2022 • edited — Member

This PR adds the initial proposal.

Before this is merged, we need to seek *approval* from the various runtimes represented in this initial draft. The following list matches the list of keys included in this initial draft. Each runtime is either linked to a discussion/issues/post requesting approval from the relative project, or someone apart of WinterCG who can represent the runtime is tagged.

- [x] Cloudflare workerd - @jasnell
- [x] Deno Land deno - @lucacasonato
- [x] Meta React Native
- [x] Netlify - @ascorbic
- [x] Electron - reached out by @Ethan-Arrowood
- [x] Node.js
- [x] Oven Bun - reached out by @Ethan-Arrowood
- [x] Vercel Edge Light - @Ethan-Arrowood

Source: https://github.com/WinterTC55/runtime-keys/pull/2

October 2022

How it worked

Keys tracked directly in document

Updates made via PRs

Approval required by group members



W3C Community Group Draft Report

TABLE OF CONTENTS

1 **Introduction**
1.1 Example Usage

2 **Keys**
2.1 Alibaba Cloud - edge-routine
2.2 Cloudflare - workerd
2.3 Deno Land - Deno
2.4 Lagon - Lagon Runtime
2.5 Meta - React Native
2.6 Netlify - Edge Functions
2.7 OpenJS Foundation - Electron
2.8 OpenJS Foundation - Node.js
2.9 Oven - Bun
2.10 Vercel - Edge Light

3 **Key Management**
3.1 Adding a key
3.2 Modifying a key

Conformance
Document conventions

References
Normative References

§ 3. Key Management

§ 3.1. Adding a key

All JavaScript runtimes are welcome to create a key and add it to this list. New keys must not conflict with another existing key. Runtime entries on this list are encouraged, but not required, to participate in the community group.

To propose a new key, create a pull request in runtime-keys repository that adds the desired key to the `index.bs` file using the following format:

```
## <Runtime Organization> - <Runtime Name> ## {#<Runtime Key>}

<Runtime Description>

Key: \`<Runtime Key>\`

Website: [<Runtime Website>](<Runtime Website>)

Repository: [<Runtime Repository>](<Runtime Repository>)
```

The key should be inserted in alphabetical order based on the first character of the Runtime Organization.

Keys should meaningfully represent the relative runtime and be a simple, string-like value so it can be used in common configuration formats such as JSON and YAML.

Keys should also not conflict with existing Browserlist entries.

Entry pull requests must be approved by at least two active members of the community group that are not directly affiliated with the proposed runtime.

§ 3.2. Modifying a key

Modifying an entry requires opening a pull request and recieving two approvals from active community group members not affiliated with the relative runtime project.

Modifications can be made to aspects such as the Organization, Name, Description, Website, and Repository.

Aside from extraordinary circumstances, to prevent breaking tools built around this list, **Keys are not modifiable**.

Keys may be deprecated to indicate inactive runtime projects.

Deprecated Keys may be undeprecated and claimed by another runtime.

To deprecate or undeprecate a key, open a pull request adding or removing `<b>[Deprecated]</b>` from the Name:

```
+ ## <b>[Deprecated]</b> <Runtime Organization> - <Runtime Name> ## {#<Runtime Key>}
- ## <Runtime Organization> - <Runtime Name> ## {#<Runtime Key>}
```

Source: https://github.com/WinterTC55/runtime-keys/pull/2

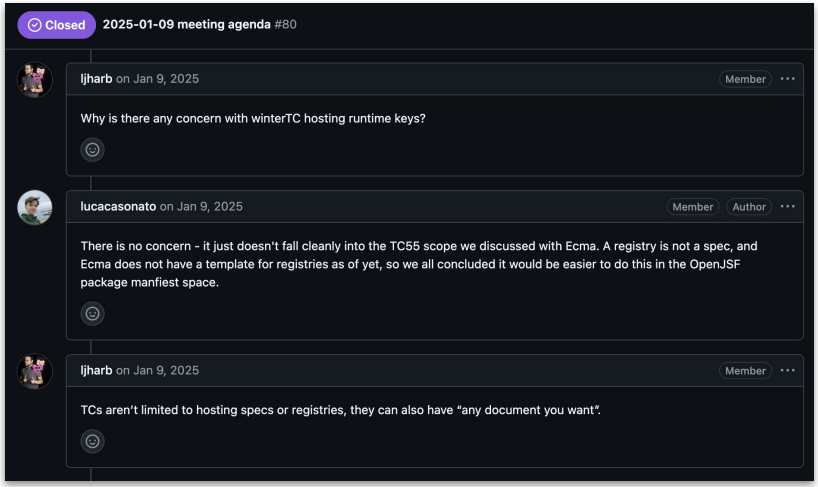
January 2025

WinterCG joins Ecma and becomes WinterTC (TC55)

What are we going to do with runtime-keys?

- Considered out-of-scope because no other WinterTC proposal normatively reference them
- Still believed to be a valuable resource
- *“...Ecma does not have a template for registries as of **yet**…”*

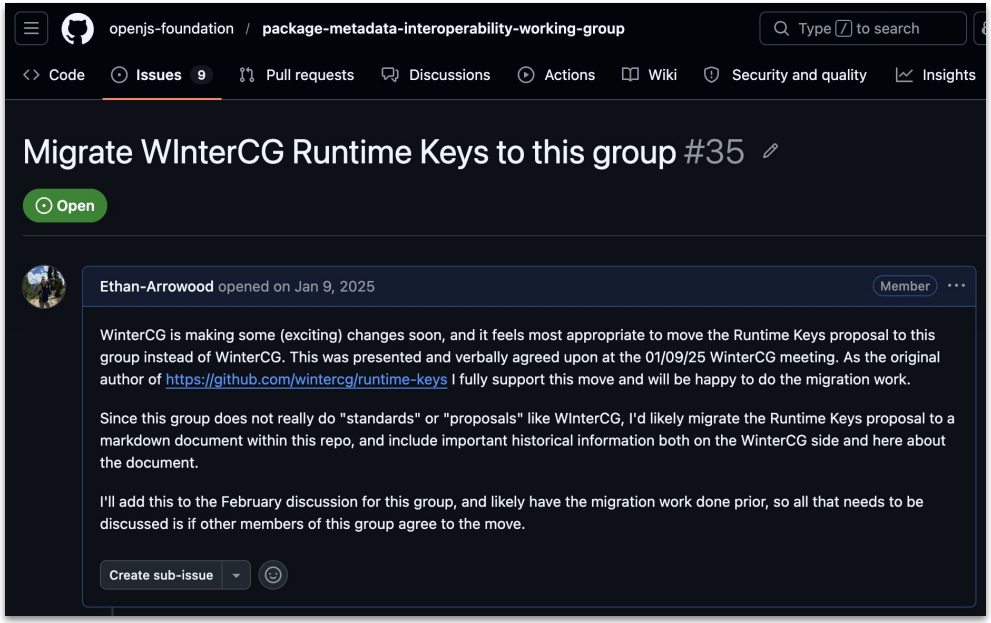




Maybe:

OpenJS Foundation Package Metadata Interoperability WG

“The goal of the Package Metadata Interoperability Working Group is to improve how JavaScript developers define their packages across the ecosystem.”



Source: https://github.com/openjs-foundation/package-metadata-interoperability-working-group/issues/35

March - June 2025

What about IANA?



Ethan-Arrowood on Mar 12, 2025 — Member — Author

I attended the latest WinterTC meeting and got some more information. Summary:

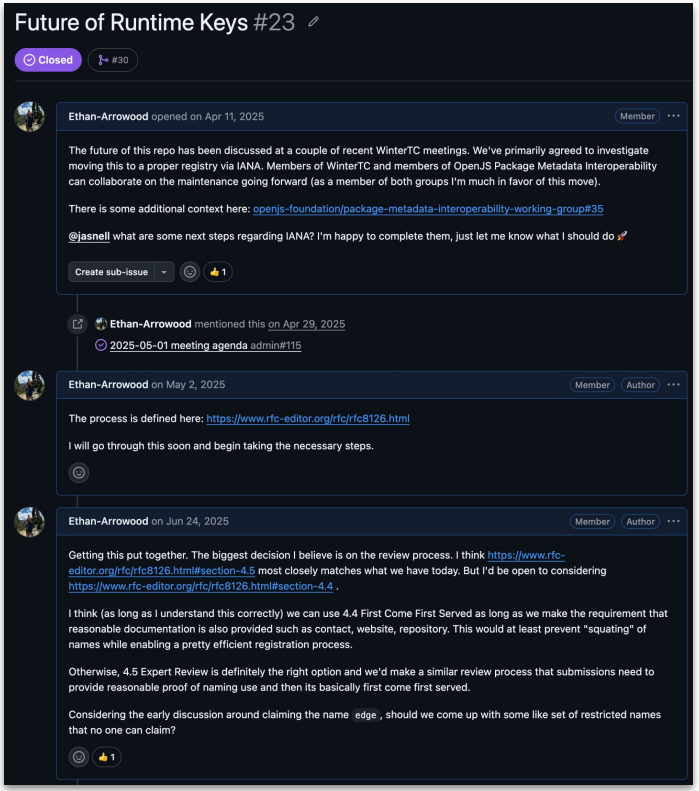
- WinterTC believes Runtime Keys is out-of-scope since no other WinterTC proposal normatively references them
  - There is a chance this changes in the future (i.e. import.meta)
- But, they believe it is still a valuable resource, and thus do not want to simply deprecate/archive.
- The recommendation is to consider transferring to a formal registry body such as IANA and it can be managed by both WinterTC and this group

Since I originally created the registry keys doc, and am actively participating in both this group and WinterTC (invited expert), I'm going to collaborate with James Snell to see what we can do about IANA.

What does everyone think? I am in full support of this and think its a much better resolution than the document being transferred to this group to maintain

Source: https://github.com/openjs-foundation/package-metadata-interoperability-working-group/issues/35

But once we started down this path, we started thinking, *what if Ecma could do this?*



Future of Runtime Keys #23

Closed — #30

Ethan-Arrowood opened on Apr 11, 2025 — Member

The future of this repo has been discussed at a couple of recent WinterTC meetings. We've primarily agreed to investigate moving this to a proper registry via IANA. Members of WinterTC and members of OpenJS Package Metadata Interoperability can collaborate on the maintenance going forward (as a member of both groups I'm much in favor of this move).

There is some additional context here: openjs-foundation/package-metadata-interoperability-working-group#35

@jasnell what are some next steps regarding IANA? I'm happy to complete them, just let me know what I should do

Create sub-issue — 1

Ethan-Arrowood mentioned this on Apr 29, 2025

2025-05-01 meeting agenda admin#115

Ethan-Arrowood on May 2, 2025 — Member — Author

The process is defined here: https://www.rfc-editor.org/rfc/rfc8126.html

I will go through this soon and begin taking the necessary steps.

Ethan-Arrowood on Jun 24, 2025 — Member — Author

Getting this put together. The biggest decision I believe is on the review process. I think https://www.rfc-editor.org/rfc/rfc8126.html#section-4.5 most closely matches what we have today. But I'd be open to considering https://www.rfc-editor.org/rfc/rfc8126.html#section-4.4 .

I think (as long as I understand this correctly) we can use 4.4 First Come First Served as long as we make the requirement that reasonable documentation is also provided such as contact, website, repository. This would at least prevent "squating" of names while enabling a pretty efficient registration process.

Otherwise, 4.5 Expert Review is definitely the right option and we'd make a similar review process that submissions need to provide reasonable proof of naming use and then its basically first come first served.

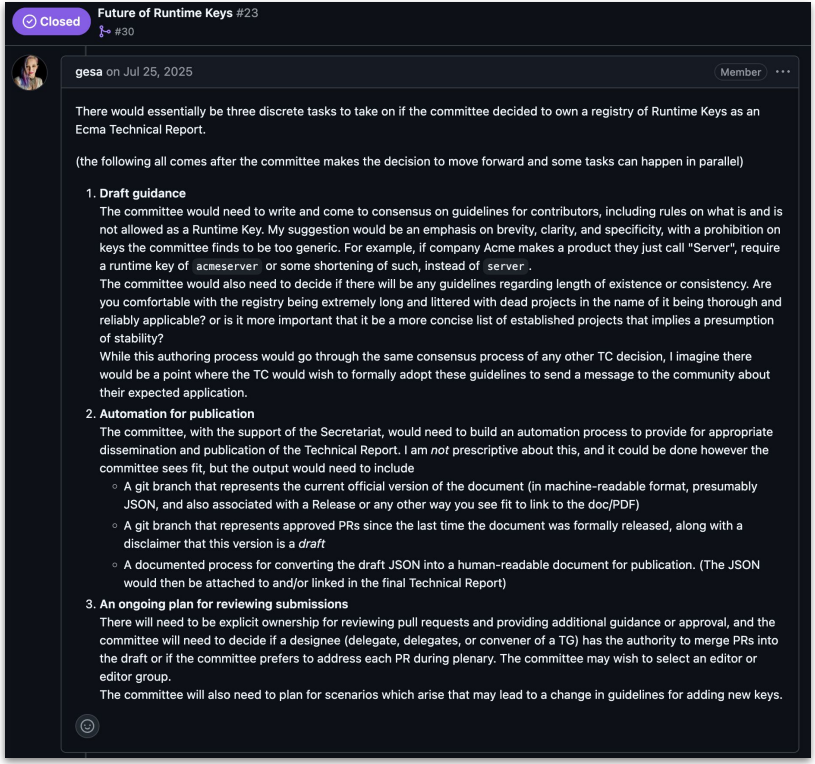
Considering the early discussion around claiming the name `edge`, should we come up with some like set of restricted names that no one can claim?

1

Source: https://github.com/WinterTC55/runtime-keys/issues/23

July 2025

Introducing: Ecma Technical Report



Closed **Future of Runtime Keys** #23

#30

gesa on Jul 25, 2025 — Member

There would essentially be three discrete tasks to take on if the committee decided to own a registry of Runtime Keys as an Ecma Technical Report.

(the following all comes after the committee makes the decision to move forward and some tasks can happen in parallel)

1. **Draft guidance**
   The committee would need to write and come to consensus on guidelines for contributors, including rules on what is and is not allowed as a Runtime Key. My suggestion would be an emphasis on brevity, clarity, and specificity, with a prohibition on keys the committee finds to be too generic. For example, if company Acme makes a product they just call "Server", require a runtime key of `acmeserver` or some shortening of such, instead of `server`.
   The committee would also need to decide if there will be any guidelines regarding length of existence or consistency. Are you comfortable with the registry being extremely long and littered with dead projects in the name of it being thorough and reliably applicable? or is it more important that it be a more concise list of established projects that implies a presumption of stability?
   While this authoring process would go through the same consensus process of any other TC decision, I imagine there would be a point where the TC would wish to formally adopt these guidelines to send a message to the community about their expected application.
2. **Automation for publication**
   The committee, with the support of the Secretariat, would need to build an automation process to provide for appropriate dissemination and publication of the Technical Report. I am *not* prescriptive about this, and it could be done however the committee sees fit, but the output would need to include
   - A git branch that represents the current official version of the document (in machine-readable format, presumably JSON, and also associated with a Release or any other way you see fit to link to the doc/PDF)
   - A git branch that represents approved PRs since the last time the document was formally released, along with a disclaimer that this version is a *draft*
   - A documented process for converting the draft JSON into a human-readable document for publication. (The JSON would then be attached to and/or linked in the final Technical Report)
3. **An ongoing plan for reviewing submissions**
   There will need to be explicit ownership for reviewing pull requests and providing additional guidance or approval, and the committee will need to decide if a designee (delegate, delegates, or convener of a TG) has the authority to merge PRs into the draft or if the committee prefers to address each PR during plenary. The committee may wish to select an editor or editor group.
   The committee will also need to plan for scenarios which arise that may lead to a change in guidelines for adding new keys.

Source: https://github.com/WinterTC55/runtime-keys/issues/23

What’s a technical report?

ecma INTERNATIONAL

Standard ECMA-262

16th edition / June 2025

ECMAScript® 2025 Language Specification

Search...

Table of Contents

Introduction
1 Scope
2 Conformance
3 Normative References
4 Overview
5 Notational Conventions
6 ECMAScript Data Types and Values
7 Abstract Operations
8 Syntax-Directed Operations
9 Executable Code and Execution Con...
10 Ordinary and Exotic Objects Beha...
11 ECMAScript Language: Source Text
12 ECMAScript Language: Lexical Gra...
13 ECMAScript Language: Expressions
14 ECMAScript Language: Statements...
15 ECMAScript Language: Functions ...
16 ECMAScript Language: Scripts and...
17 Error Handling and Language Exte...

ECMAScript® 2025 Language Specification

ecma INTERNATIONAL

About this Specification

The document at https://tc39.es/ecma262/ is the most accurate and up-to-date ECMAScript specification. It contains the content of the most recent yearly snapshot plus any finished proposals (those that have reached Stage 4 in the proposal process and thus are implemented in several implementations and will be in the next practical revision) since that snapshot was taken.

ecma INTERNATIONAL

Stand

ECMA
Lang

ecma INTERNATIONAL

Technical Report

ECMA TR/104

1st Edition / December 2011

ECMA-262 Test Suite

Wait hold on what does “normative” even MEAN?

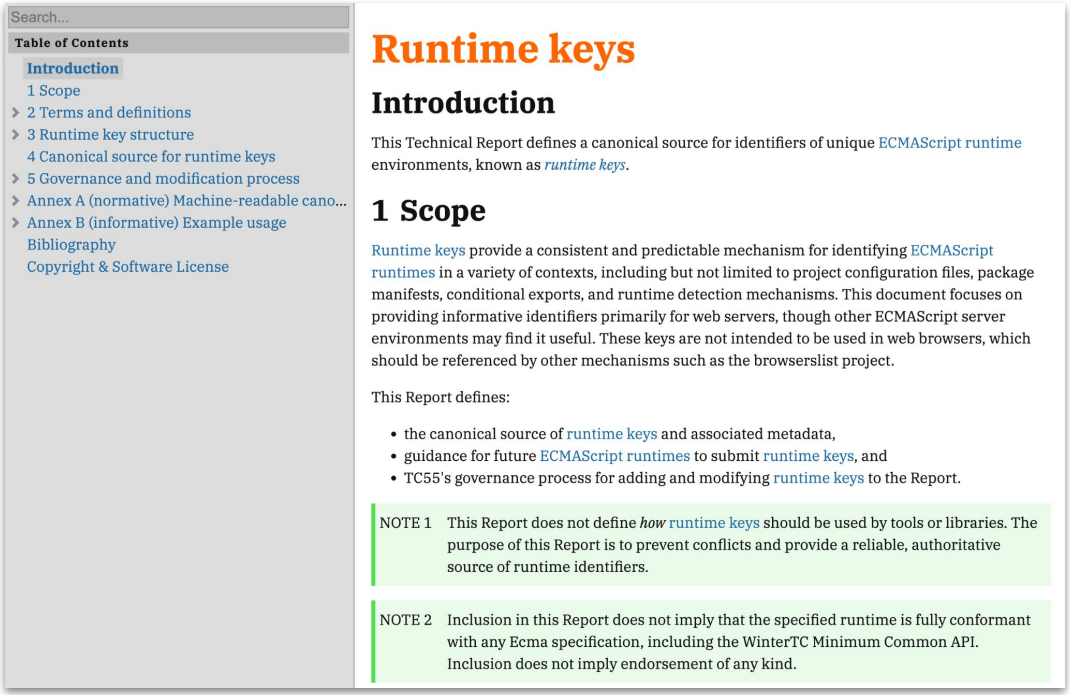
the pitch

How may it be pulled off?

- Git as single source of truth
- Produce & maintain machine-readable source
- Structured to include normative & Annex/non-normative (such as retired/archived keys)
- CD/GH Actions to generate *Draft* TR with every merge to draft branch
- Quarterly or semiannual publication of document generated by CD (merge draft to main)
- Mirror to Git service in Switzerland

January - April 2026

Implementing Runtime Keys Ecma Technical Report



https://github.com/WinterTC55/runtime-keys/pull/30
https://runtime-keys.proposal.wintertc.org/

Implementing Runtime Keys Ecma Technical Report

Use PRs to create and modify keys

Committee approves at meetings

Draft report reflects decisions immediately

Report is formally published at twice-annual Ecma General Assembly

5 Governance and modification process

5.1 Proposing new runtime key entries

All ECMAScript runtimes are welcome to propose a key for inclusion in this Report, subject to the structure as described in 3. Runtimes which contribute to this Report are encouraged, but not required, to participate in Ecma TC55.

5.1.1 Proposal process

1. The proposer creates a pull request in the WinterTC55/runtime-keys GitHub repository.
2. The pull request adds the runtime entry to the machine-readable source data file (`runtime-keys.json`).
3. The entry includes all mandatory metadata fields as described in 3.5.
4. The entry is inserted in alphabetical order by key.
5. The pull request is reviewed by TC55 participants.
6. At least two delegates within TC55 not directly affiliated with the proposed runtime approve the pull request.
7. The pull request is formally approved at a TC55 plenary meeting before merging.
8. After plenary approval, designated editors merge the pull request and update the current draft Report.

5.1.2 Expert review

All proposed entries undergo expert review to ensure compliance with the naming guidelines specified in 3.3 and other expectations. Reviewers evaluate whether:

- the proposed key is appropriately specific and non-generic,
- the runtime has demonstrated proof of use or adoption,
- the key does not conflict with existing entries or Browserslist identifiers, and
- the metadata is complete and accurate.

TC55 may reject a runtime key submission for any reason, including those not enumerated in the above sections. In such a scenario, the Committee will provide that reason and information on how to resolve any issue.

Implementing Runtime Keys Ecma Technical Report

Report is backed by a JSON data set

Great for programmatic usage both for managing the TR itself, and for downstream consumers.

Annex A (normative) Machine-readable canonical source

The authoritative, machine-readable form of the runtime key source is maintained as a JSON file: **`runtime-keys.json`**.

A.1 JSON schema

The runtime keys source data JSON file conforms to the following structure:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Runtime Keys",
  "description": "A collection of runtime keys for ECMAScript runtime environments",
  "version": "1.0.0",
  "lastModified": "YYYY-MM-DD",
  "runtimes": [
    {
      "organization": "string (required)",
      "name": "string (required)",
      "key": "string (required)",
      "description": "string (required)",
      "website": "string | null (optional)",
      "repository": "string | null (optional)",
      "deprecated": "boolean (required, default: false)"
    }
  ]
}
```

A.2 Accessing the source data

The current machine-readable source is available at:

- Draft version: **`https://github.com/WinterTC55/runtime-keys/blob/main/runtime-keys.json`**
- Published version: To be determined upon Ecma TR publication

Thank you! Questions?



Ethan Arrowood ethan@arrowood.dev

Head of Open Source Engineering@ Harper (https://harper.fast)

Website: https://ethanarrowood.com

GitHub: https://github.com/Ethan-Arrowood

BlueSky: https://ethan-arrowood.bsky.social



Aki Rose Braun aki.braun@ecma-international.org

Senior Manager, Technical Programs and Partnerships Ecma International

bsky: @akiro.se

masto: aki@toot.cat